

Insights from 15 Years of ATAM Data: Towards Agile Architecture

Authors:

Stephany Bellomo, Software Engineering Institute Carnegie Mellon University (SEI/CMU), Ian Gorton, SEI/CMU, Rick Kazman, SEI/CMU and Univ. of Hawaii

Abstract

Agile teams strive to balance short term feature development with longer term quality concerns. These evolutionary approaches often hit a "complexity wall" from the cumulative effects of unplanned changes, resulting in unreliable, poorly performing software. Consequently, there is renewed focus on approaches to address architectural concerns within the Agile community. We present an analysis of quality attribute concerns from 15 years of Architecture Trade-off Analysis Method data, gathered from 31 projects. We found that modifiability is the dominant concern across all project types; additionally there was considerable focus on performance, availability, and interoperability. For information technology projects, a relatively new quality—deployability—has emerged as a key concern. Our results provide insights for Agile teams allocating architecture-related tasks to iterations. For example they can use these results to create checklists for release planning or retrospectives to help assess whether a given quality should be addressed to support future needs.

Key Terms: Architecture evaluation, Agile, technical debt, incremental development, modifiability, deployability

Introduction

Due to concerns about the long-term deterioration of quality of large-scale Agile projects (termed “technical debt” [1]), there is renewed interest in software architecture within the Agile community [2][3] [4]. This interest indicates growing consensus that a strong architecture foundation enables teams to rapidly evolve complex software systems using iterative, incremental development approaches. This position is consistent with the arguments made in IEEE Software Magazine’s special issue on agility vs. architecture [12].

One of the major challenges Agile teams face in building an architecture foundation is balancing the competing concerns of delivery of near-term functional requirements (based on the Agile philosophy of delivering user value early and often) and near and long term quality attribute goals (without which the project can grind to a halt as system complexity makes efficient modifications impossible). In particular, quality attribute (QA) prioritization can be especially difficult in early increments, and a wrong decision can have serious ramifications resulting in hard-to-modify, unreliable, slow, or insecure systems [5].

To this end, we have sought to obtain empirical evidence on the most common QA concerns that projects must address, and their relative importance. We performed a meta-analysis on the results of ATAM analyses carried out by the Software Engineering Institute over the last 15 years. The ATAM leverages QA scenarios identified and prioritized by the project stakeholders and uses this information to evaluate the architecture and identify architectural risk (a brief description of ATAM is provided in the sidebar). In this article, we examine the results from this ATAM data, and describe how these results can be used by Agile teams for QA prioritization and allocation.

Sidebar: The ATAM

The ATAM is a method for analyzing software and system architectures with respect to quality goals [7]. The ATAM has a long pedigree—it has been used in many government and commercial organizations for more than 10 years—and has a substantial amount of supporting documentation in the form of books, papers, and training courses. The ATAM is traditionally enacted in two phases. Phase 1 is focused on eliciting information about the architecture from the architecture team. Phase 2 is focused on eliciting project stakeholder needs. The ATAM includes many component techniques that have been developed and refined over the years such as **Quality Attribute Scenario (QAS) analysis** which is a method for capturing architecturally focused requirements from stakeholders. QAS follow a structured format defined in 6 parts: 1. an explicit *stimulus*, 2. a *stimulus source*, an explicit *response*, 3. a *response measure*, 4. the *environment*, and 5. the *artifact* affected by the scenario.. The **utility tree** is a hierarchically organized affinity grouping of quality attribute concerns/ scenarios derived from a particular project [7]. **Quality attribute tactics** are architectural design primitives that ATAM evaluators look for in the architecture, to quickly assess the strategies employed by the architects [7]. These component techniques aid the ATAM team in repeatable eliciting and analyzing QA information.

ATAM Study Results

We present results from two separate studies of ATAM data collected from 31 projects. In Study 1 Ozkaya, Nord, Bass, and Sangwan analyzed ATAM scenario data from 1999 to 2006 [6]. In Study 2, we analyzed ATAM scenario data from 2006 to 2013. The QA scenario data examined in both studies was generated as part of the Phase 1 of the ATAM, collected from the elicited utility trees (as described in [7]).

The method we used for our analysis is described in Sidebar: Analysis Methodology. ATAMs conducted by the SEI cover a wide range of project types across government and industry domains. In Table 1, we have categorized the data into IT systems versus embedded systems, and government versus non-government projects. The details of Study 1's project profile are described in [6]. Study 2 includes nine projects and 348 QA scenarios and projects in Study 2 were more heavily weighted toward IT projects (78% versus 42%)

Sidebar: Analysis Methodology

Our analysis was conducted in four structured stages using three independent "coders" ("coders" are analysts assigning tags, category designations, to scenarios).

- **Stage 1:** Collected ATAM reports 2006-2013 and consolidated these into a single data set.
- **Stage 2:** Independent coding of a sample of the data (100 records) using a shared set of tag values to assess and address variability in across analysts. This stage culminated in a session reconciling coding discrepancies before moving on to stage 3.
- **Stage 3:** Independent coding of all of the remaining records in the data set and final reconciliation.
- **Stage 4:** Categorization, summation, and analysis of the full data set, producing the results summarized this article.

Study Title	Study Timeframe	% IT	% Embedded	% Government	% Non-Government
Study 1	1999-2006	58%	42%	88%	12%
Study 2	2006-2013	78%	22%	66%	34%

Table 1: ATAM project profiles by study

In Figure 1, we compare QA scenario analysis results from 1999 to 2013 ATAM data for both Study 1 and Study 2. Categories that contained less than three scenarios were not included in the results shown in Figure 1. Based on our analysis, we found the top QA concerns to be consistent. For example, modifiability was the most common QA across both studies. Performance was the second most important QA in Study 1 and third (by a small margin) in Study 2. This suggests that while the technical environment for software-intensive systems has evolved over the years, the most important stakeholder and architectural concerns have changed little.

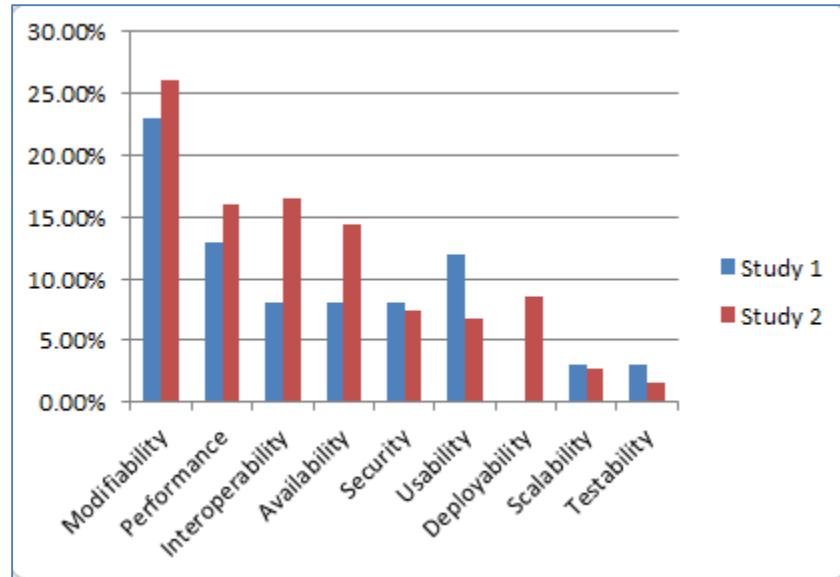


Figure 1: Quality Attribute Summary of Study 1 and Study 2 Data

In Figure 2, we take a more detailed look into Study 2 QA data that covers ATAM data from 2006 to 2013. This table lists the top 20 specific QA concerns ranked from highest to lowest, based on frequency of occurrence in our data set. This view of QA concern data reinforces the importance of Modifiability, Performance, Interoperability, Availability (and, to a lesser extent, Usability), but also suggests that project teams prioritize specific QA concerns in a finer-grained way than what was shown in Figure 1. For example, consider the top three QA concerns: “Modifiability: Reduce coupling” is followed by “Performance: Latency”, which is in turn followed by “Interoperability: Upgrade and Integrate with other system components”.

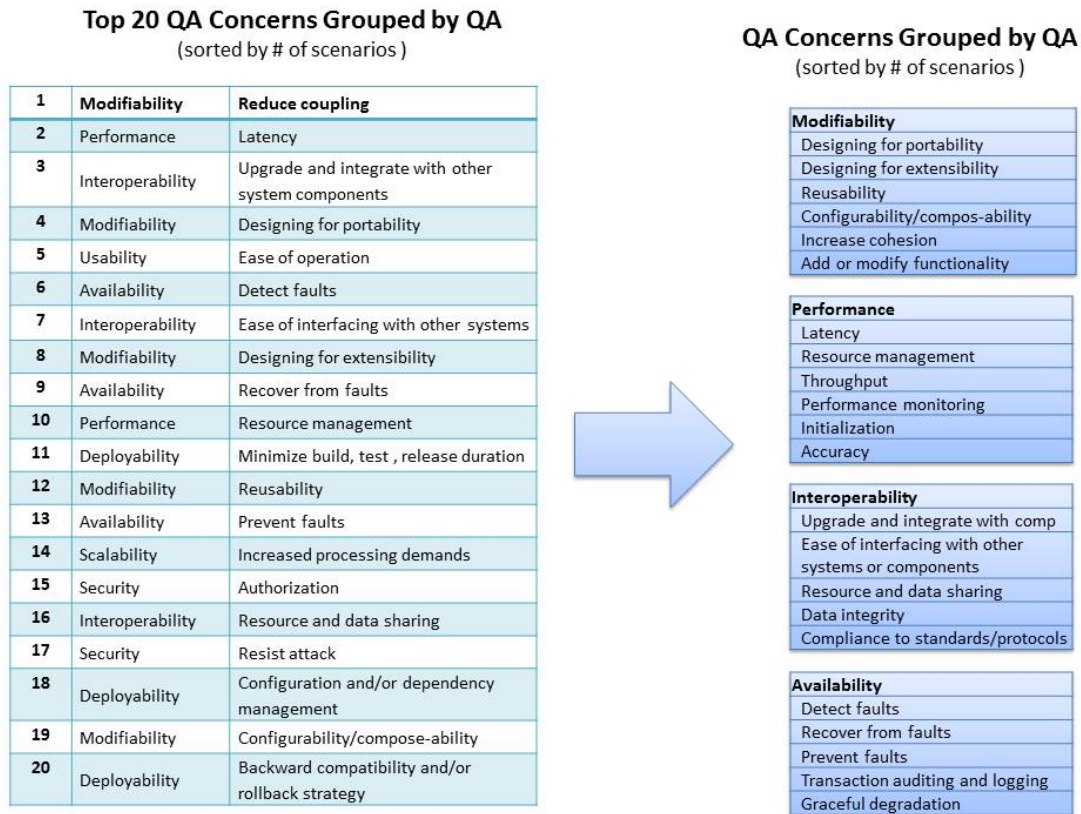


Figure 2: Top 20 Study 2 Quality Attribute Concerns

Data Analysis and Discussion

Below we discuss some of the implications of this data.

Modifiability leads. In Figure 1, *Modifiability* ranks highest across both studies (24% if all scenarios for Study 1, 26% for Study 2), and in Figure 2, *Modifiability*-related concerns represent five of the top twenty QA concerns. For this reason, we performed a detailed analysis of the modifiability concerns in the ATAM data set. Summarizing modifiability scenarios and the resulting data is shown in Table 2 (the numbers represent, for each Modifiability-related concern, its percentage of Modifiability scenarios). The top two concerns in Table 2 comprise 47% and focus on anticipating changes in software. A variety of system types are represented in the ATAM data and anticipated change examples include adding or replacing 3rd party components,

Summary of Modifiability Scenario Data	#
Add, replace, modify functionality or component (e.g., 3rd party component, new sensor, adaptor, algorithm)	32%
Configurable software (e.g., order throughput, UI content, display, feature toggle, reporting, simulation)	15%
Operating system or registry changes	11%
Replace or change networked environment (e.g., new protocol, new network platform)	10%
Change, add or replace model or tool capability (runtime and non-runtime)	8%
Software component composition or reuse (from core asset base)	7%
Port or integrate new technology, device or HW platform (e.g., new vehicle, device, sensors, dual core)	6%
Changes to message content and formats (translation may be required)	4%
Add or replace services (e.g. messaging)	3%
User or context-driven adaptation (e.g., ActiveX not allowed)	
Replace or upgrade middleware, webserver or database	2%
Minimize merge complexity	2%

Table 2: Summary of Modifiability Scenarios

adding new sensor types, replacing algorithms, and changing UI content. However, the scope of stakeholder concern for change is not limited to the software application alone. A significant number of the scenarios focus on changes external to the software such as operating system replacement, network modifications, new technology integration, hardware platform upgrade, and so on. A list such as this, derived from real project data, can help teams reason about future changes they should consider and avoid being caught off guard by an impactful unanticipated change.

Performance, Interoperability, Availability. Referring back to Figure 1, we observe that in addition to Modifiability, *Performance*, *Interoperability* and *Availability* were all frequently occurring qualities. In Table 3 below, we summarize and rank the Performance and Availability concerns. A common theme emerges when we look at the Performance and Availability scenarios side-by-side. A majority of these scenarios focus on maintaining critical capability and consistent response time in the face of partial or major system failure (80% of Performance scenarios, 64% of Availability scenarios). Examples of failure scenarios shown in Table 3 include situations such as loss of hardware, software failure, network outage/failure, dramatic increase in load, and synchronization failure. We suggest that in addition to *Modifiability*, these types of failure scenarios should be considered by teams in reasoning about work they may need to plan for (assuming they want to avoid these types of scenarios).

Summary of Performance Scenario Data		Summary of Availability Scenario Data	
Maintain response time for a system capability (e.g., simulation, display, backup, run report)	25%	Hardware failure (or access blocked) and critical operational capabilities remain intact (e.g. server, network card)	34%
Maintain response time for critical capabilities with limited and/or intermittent resource	25%	Detect faults, update log, monitor outage trends (e.g., software component failure, bad node)	19%
Maintain response time while hardware or platform changes (e.g., different machine, new CPU architecture)	21%	Software failure or bad data input and critical operational capabilities remain intact	15%
Maintain response time while load or usage increases (e.g., increase load, increase service demands, increase fidelity)	9%	Restart after failure without loss of data or state information (e.g., rollback, resynchronize)	11%
Maintain startup and/or shutdown response time threshold	8%	Network or messaging failure but critical operational capabilities remain intact	9%
Monitor system or network performance	6%	Synchronization failure (e.g., clock time, data sources) but critical operational capabilities remain intact	6%
Process transactions with increased load within acceptable degraded range (e.g., increased transactions, larger file size)	4%	Visible notification to user if HW or SW components fail	6%
System capability works as expected	2%		

Table 3: Summary of Performance and Availability Scenario Data

Comparing IT versus Embedded Concerns (Study 2). We also explored differences in QA concerns in Study 2 between information technology (IT) projects and embedded systems, as shown in Figure 3.

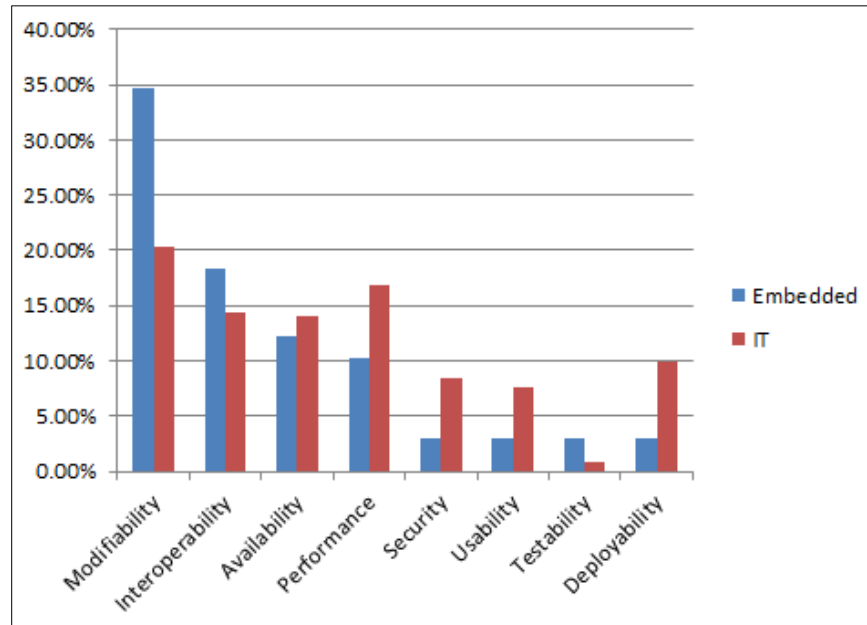


Figure 3: IT versus Embedded System Quality Attributes (Study 2)

One stark difference is the importance of *Modifiability* for embedded projects, which accounts for almost 35% of all of the identified QA concerns in the embedded grouping - 15% greater than any other QA category. In contrast, while *Modifiability* is also the most important concern for IT projects, there is not as much of a gap between *Modifiability* and the other QAs. The majority of the IT project QA counts, including *Modifiability*, range between 14% to 20%. We suggest that these numbers could reflect either the underlying QA requirements of the system or the current perceived problems of the design. If these results reflect underlying QA requirements, this finding suggests that for embedded projects it is important to consider requirements that promote portability and encapsulation to reduce the impact of changes to external subsystems (e.g. hardware, networks) over time. In the case of IT projects, the more even spread across several QAs such as *Modifiability*, *Security*, and *Usability* suggests a need for balancing requirements considerations more broadly. While there is no one-size-fits-all QA list of concerns, we propose that these project-specific views can be useful when considering how to prioritize in an IT versus embedded context.

Deployability as an emerging concern. In Figure 3 it can be seen that *Deployability* represents 10% of the QA concerns for Study 2 IT projects. This concern has emerged in recent years (it is *only* found in 2006-2013 data) so we decided to take a closer look at *Deployability* QA scenarios. Examples of *Deployability* scenarios from ATAM data are shown in Table 4. While these concerns cross-cut many existing QA areas, such as *Modifiability*, we found that goals driving *Deployability* are different, as are the response measures. For example in DS-1 to DS-4 the driving goal is towards reducing complexity and reducing deployment and testing cycle time. D5 and D6 emphasize minimizing ripple effects due to change; however, the overarching stakeholder concern is change management for a complex distributed deployment environment. Reducing deployment and automated testing cycle time is a focus for books on this topic, such as Humble and Farley's, *Continuous Delivery* [8]. These scenarios may give teams a starting place for reasoning about scenarios to enable their deployment goals.

Example	Deployability Example Scenario
DS-1	Plan for a new release, the final upgrade plan needs to be developed within a week. (a lot of time is spent on specific installation's configurations).
DS-2	There is an upgrade being pushed out, the software is robust enough to handle 99% errors that occur and leave 1% to be handled manually.
DS-3	Complete all full controller builds in < 5 minutes (even under peak load of the build farm).
DS-4	To create/build a new system-level test case, the average time to build the test case is one day.
DS-5	Co-developers make modifications to release version x while the other team is making architectural changes to version x and release version x+1 that isn't synch'ed with the co-developers changes. The co-developer integrates the modifications in version x+1 within 8 weeks.
DS-6	There is a new version of the spec that is not compatible. Software application and the adapter should be backwards compatible, in addition to supporting the new version.

Table 4: Deployability Example Scenarios

Making Use of the Data

The data and concrete examples presented in this paper can be used to help Agile teams reason about which qualities they should focus on in *early* increments and when planning future increments. High ranking QAs and QA concerns (including examples) can be incorporated as checklists at designated points during the incremental lifecycle to help teams quickly assess whether they need to consider a specific QA concern to support an anticipated business need.

Evolving a system's architecture in an Agile project presents unique software engineering challenges.. To provide assistance, we suggest that QA concerns can be woven into the lifecycle that a project team is using and furthermore that this provides natural opportunities to weave in the considerations presented here. For example, during Agile retrospective meetings the team may brainstorm about QA-related deficiencies that have surfaced in the prior increment. During sprint planning the architect may propose addressing a QA concern, adding it to the sprint backlog if the product owner agrees. This data also provides ammunition for the Agile team lead in making the business case to the product owner, justifying the importance of dealing with such concerns architecturally.

The data shown on the right side of Figure 2 can also serve as checklist for team members as they reflect on QA areas of concern. Finally, this data can help Agile developers in creating review questions to reason about design trade-offs such as, "Did we negatively impact another architecturally significant QA concern with this design decision?" An example of applying QA concern and design tactic data in this way is described in [10].

Our ultimate aim is to provide systematic support for iteratively addressing architectural concerns in Agile projects. Techniques for efficiently identifying, prioritizing and allocating architecture tasks to iterations, and measuring their on-going effects on project velocity are fundamental to the success of this work. With respect to future applicability, data such as this could also be used as input to decision

support system models as described in [11] which provide automated support for reasoning about QA tradeoffs.

Acknowledgements:

This material is based upon work funded and supported by the Department of Defense under Contract No. FA8721-05-C-0003 with Carnegie Mellon University for the operation of the Software Engineering Institute, a federally funded research and development center. This material has been approved for public release and unlimited distribution. DM-0002047

References

- [1] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, R. Sangwan, C. Seaman, K. Sullivan, N. Zazworka, "Managing Technical Debt in Software-Reliant Systems", *Workshop on the Future of Software Engineering Research at ACM SIGSOFT FSE-18*, (Santa Fe, NM), November, 2010.
- [2] S. Ambler, *Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise*, IBM Press, 2012.
- [3] D. Leffingwell, *Scaling Software Agility: Best Practices for Large Enterprises*, Addison-Wesley, 2007.
- [4] S. Bain, *Emergent Design: The evolutionary nature of Professional Software Development*, Addison-Wesley, 2008.
- [5] S. Bellomo, R. Nord, and I. Ozkaya. *A Study of Enabling Factors for Rapid Fielding; Combined Practices to Balance Tension between Speed and Stability* Proceedings of International Conference on Software Engineering 2013.
- [6] I. Ozkaya, L. Bass, R. Nord, R. Sangwan, "Making Practical Use of Quality Attribute Information", *IEEE Software*, March/April 2008.
- [7] P. Clements, R. Kazman, M. Klein, *Evaluating Software Architectures: Methods and Case Studies*, Addison- Wesley, 2001.
- [8] J. Humble and D. Farley, *Continuous Delivery*, 1st ed., Chapter 4, Addison Wesley, August 2010.
- [9] F. Bachmann, L. Carballo, J. McHale, R. Nord, "Integrate End to End Early and Often", *IEEE Software*, July/August 2013.
- [10] J. Ryoo, P. Laplante, R. Kazman, "In Search of Architectural Patterns for Software Security", *IEEE Computer*, June 2009, 42:6, 98-100.
- [11] N. Ernst, I. Gorton: Using AI to model quality attribute tradeoffs. *AIRE 2014*: 51-52
- [12] *IEEE Software*, "Special Issue: Agility vs. Architecture", March/April 2010 vol. 27 no. 2

Author Bios



Stephany Bellomo is a senior member of the technical staff on the Carnegie Mellon Software Engineering Institute's Architecture Practices team. Current interests include research in architecting for agility, technical debt, and DevOps/Continuous Delivery. Stephany served on the organizing committee for the 2014 International Workshop on Release Engineering and is currently serving as guest editor of IEEE Software Magazine 2015 Special Issue on Release Engineering. She has been an SEI Software Architecture Conference program committee member since 2010. Stephany teaches courses in Software Architecture and Service-Oriented Architecture. She has an MS in Software Engineering from George Mason University. She is a Senior Member of the IEEE Computer Society.



Dr. Rick Kazman is a professor at the University of Hawaii and a Principle Researcher at the Software Engineering Institute. His primary research interests are software architecture, design and analysis tools, software visualization, and software engineering economics. He is the author of over 150 papers and co-author of several books, including *Software Architecture in Practice* and *Evaluating Software Architectures: Methods and Case Studies*. Kazman was one of the creators of the SAAM (Software Architecture Analysis Method) and the ATAM (Architecture Tradeoff Analysis Method). Dr. Kazman received a Ph.D. from Carnegie Mellon University. He is a Senior Member of the IEEE Computer Society.



Ian Gorton is a senior member of the technical staff on the Carnegie Mellon Software Engineering Institute's Architecture Practices team, where he investigates issues related to software architecture at scale. This includes designing large-scale data management and analytics systems and understanding the inherent connections and tensions between software, data, and deployment architectures. Gorton received a PhD in computer science from Sheffield Hallam University. He is a senior member of the IEEE Computer Society.

Author Contact Info:

Stephany Bellomo sbellomo@sei.cmu.edu, Ian Gorton igorton@sei.cmu.edu, Rick Kazman kazman@sei.cmu.edu

